Inginerie Software

Laborator 5. Borland Together Architect

Borland Software Corporation a fost fondata in 1983 de Niels Jensen, Ole Henriksen, Mogens Glad si Philippe Kahn. Activitatea sa principala a fost dezvoltarea si vanzarea de produse software de dezvoltare si implementare software. Borland a avut prima sediu in Scotts Valley, California, apoi in Cupertino, California si apoi in Austin, Texas. in 2009, compania a devenit o filiala completa a firmei britanice *Micro Focus International plc.*
Dintre numeroasele produse ale companiei amintim:

- Borland C++
- C++Builder
- C#Builder
- JBuilder
  Borland AppServer
- Borland Enterprise Server
- Borland Enterprise Studio, for C++, Mobile and Java
- Borland Together for Eclipse, s.a.
-

Borland Together for Eclipse, cunoscut astazi sub denumirea de Together este un produs de la Micro Focus, anterior de la Borland, fost de la TogetherSoft (achizitionat de Borland in 2003), care integreaza in prezent un IDE Java, care initial isi avea radacinile in Jbuilder, cu un instrument de modelare UML.

Linia de produse a fost vanduta anterior in trei niveluri de functionalitate: **Together Developer**, **Together Designer** si **Together Architect**; totusi, din 2007, Together a fost unificat intr-un singur produs. Versiunile anterioare ale produselor Together erau aplicatii autonome complet proprietare, in timp ce incepand cu versiunea 2006, Together se bazeaza pe Eclipse.

Together este implementat ca un set de plugin-uri Eclipse, ofera modelarea Unified Modeling Language (UML) 1.4, suport multilingv, modelare fizica a datelor, modele de proiectare, recunoasterea modelelor de proiectare a codului sursa, proiectarea si reutilizarea sablonului de coduri, generarea documentatiei si audituri si metrici de coduri. Together adauga diagramarea UML 2.0, modelarea proceselor de afaceri si modelul de date logice, precum si transformarea modelului de date fizice si logice in suportul modelelor personalizate.

Together utilizeaza in prezent platforma Eclipse 4.3.1.
Diagramele BPMN (Business Process Model and Notation)pot fi create prin import si utilizate pentru a genera iesirea in limbajul de executie a procesului de afaceri cu definitii de servicii web (BPEL4WS- *Web Services Business Process Execution Language*). Auditurile si valorile sunt furnizate atat la nivel de model, cat si la nivel de cod, definit in *Object Constraint Language* (OCL) 2.0. Together accepta modele de proiectare, Java , C

++, CORBA si QVT (*Query/View/Transformation*) – standard al OMG (object Management Group.

Instalarea

1. Dezarhivati Borland Together Archtect 2006(1).zip
2. Lansati

```
.....\Borland Together Archtect 2006\Borland Together Archtect 2006\WINDOWS\TEC_ARCHITECT_2006_WINDOWS.exe
```

3. Dupa instalare urmati pasii mentionati in

```
.....\Borland Together Archtect 2006\Borland Together Archtect 2006\UPDATE\UPDATE\info.txt
```

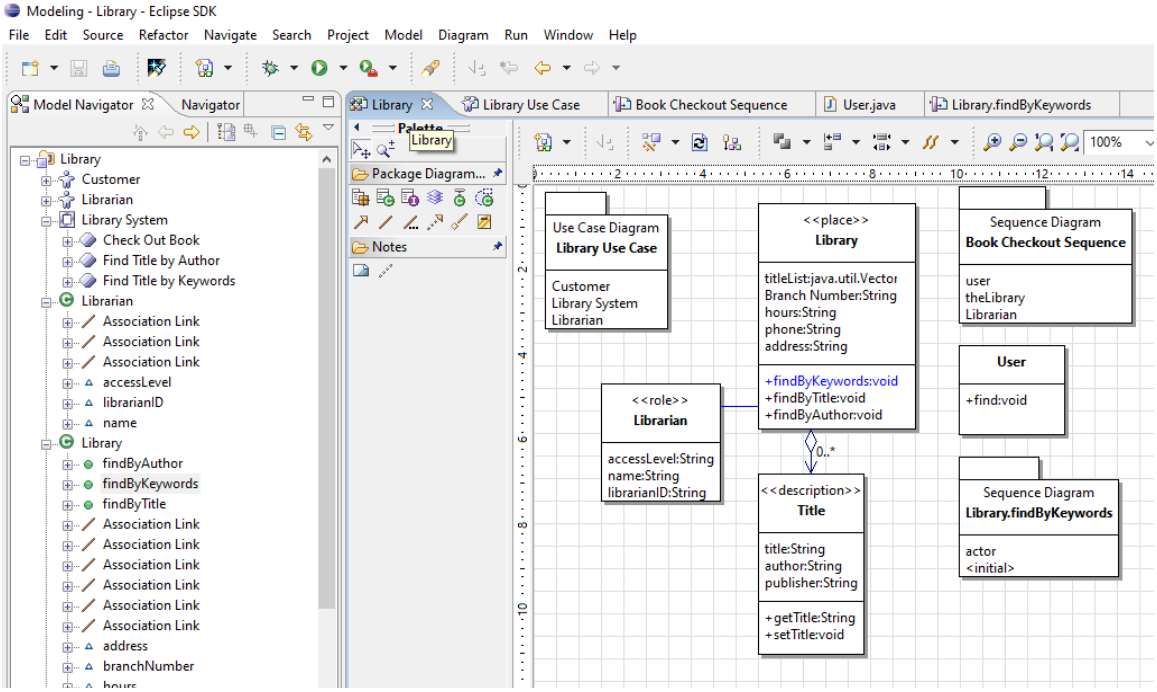
Exercitii

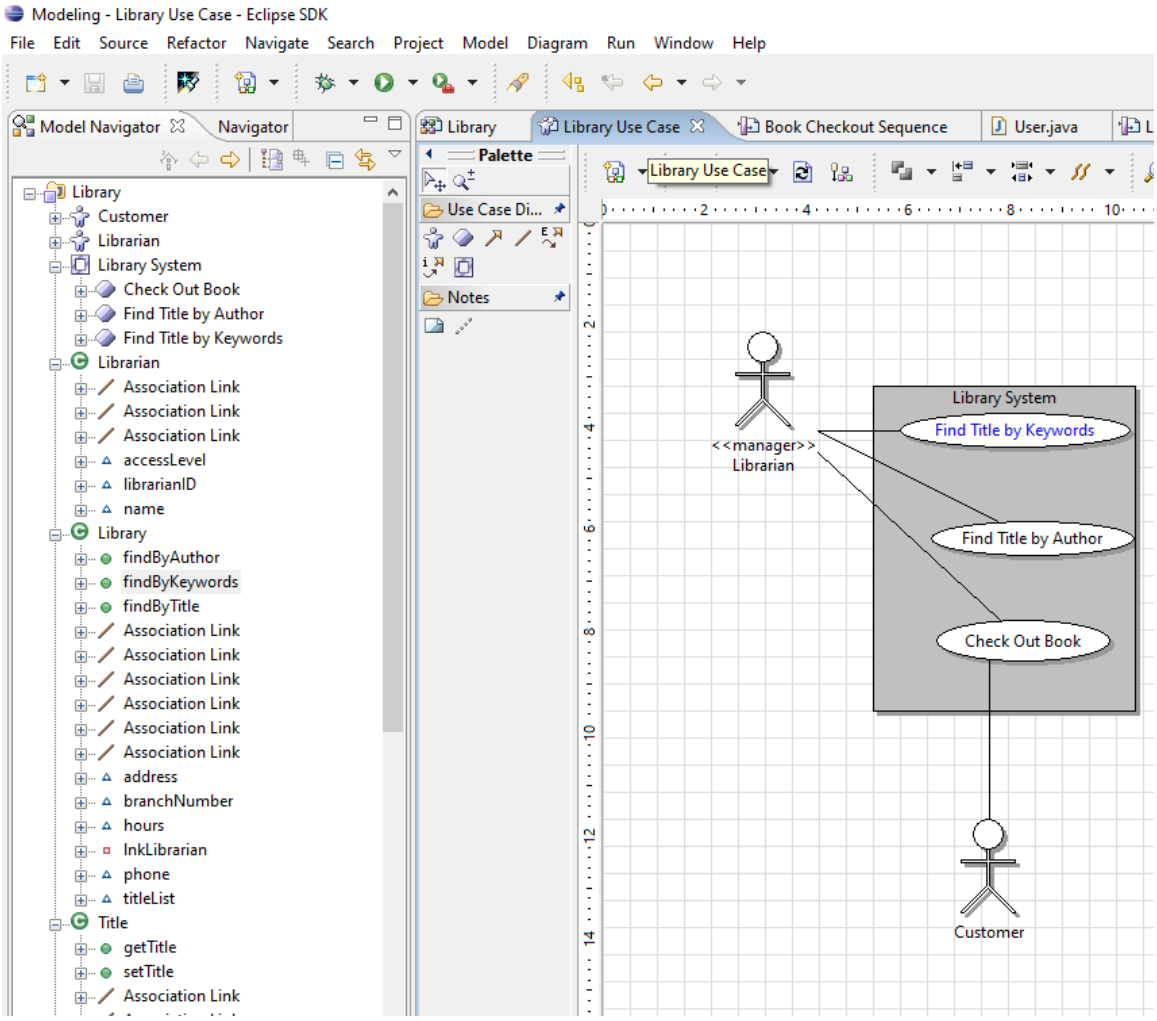
5.1. Studiati tutorialul prezent in directorul de instalare

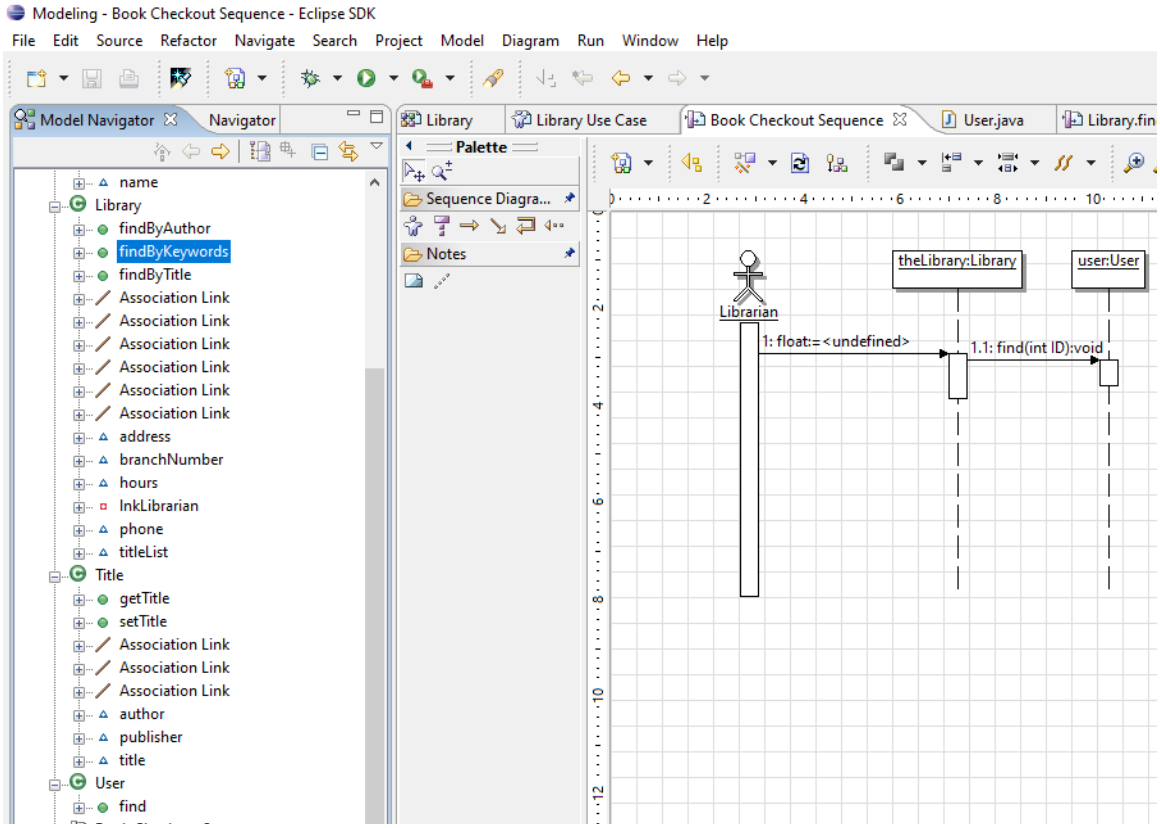
```
.....\Borland\Together\Architect2006\docs\GettingStarted.pdf
```

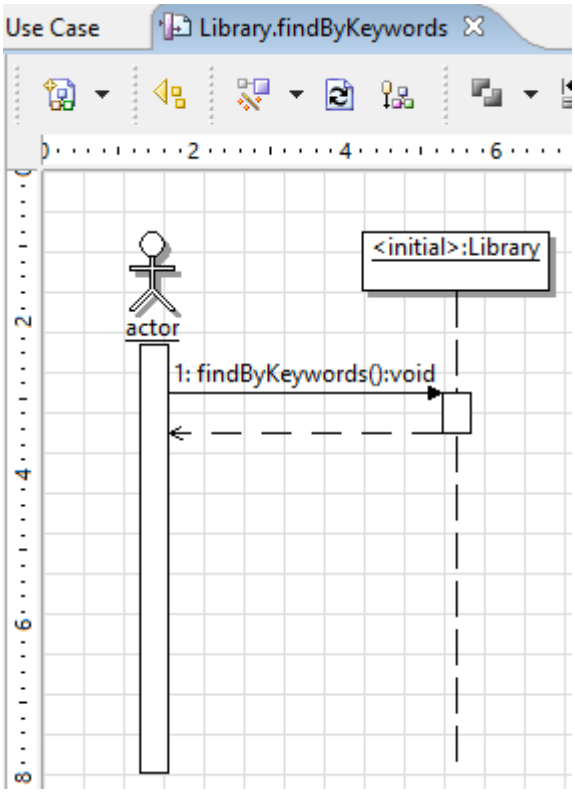
5.2. Dupa modelul *Library* - Java Modeling Project (pag 12-38), creati un proiect similar personalizat(*LibraryNumePrenume*) pentru a va familiariza cu diferitele caracteristici ale Together:

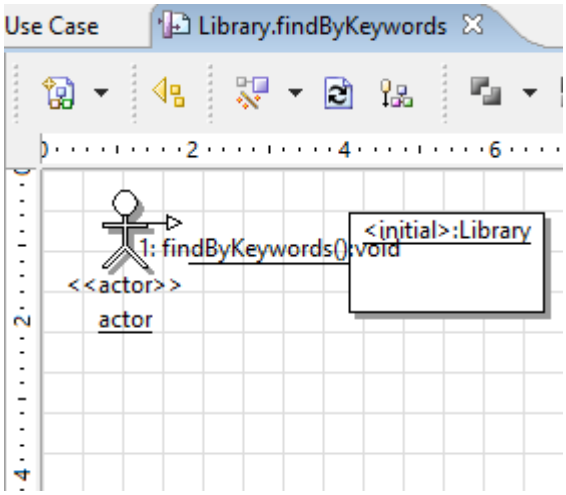
- crearea unui proiect
- lucrul cu editorul de diagrame
- crearea unei diagrame use case
- crearea unei diagrame de tip clasa
- crearea unei diagrame de tip secventa
- conversia diagrama secventa - diagrama de colaborare


Modeling - Library - Eclipse SDK
File Edit Source Refactor Navigate Search Project Model Diagram Run Window Help
Model Navigator
Navigator
Library
Customer
Librarian
Library System
Check Out Book
Find Title by Author
Find Title by Keywords
Librarian
Association Link
Association Link
Association Link
accessLevel
librarianID
name
Library
findByAuthor
findByKeywords
findByTitle
Association Link
Association Link
Association Link
Association Link
Association Link
Association Link
address
branchNumber
Library
Library Use Case
Book Checkout Sequence
User.java
Library.findByKeywords
Palette
Library
Package Diagram...
Notes
100%
Use Case Diagram
Library Use Case
Customer
Library System
Librarian
<<place>>
Library
titleList:java.util.Vector
Branch Number:String
hours:String
phone:String
address:String
+findByKeywords:void
+findByTitle:void
+findByAuthor:void
Sequence Diagram
Book Checkout Sequence
user
theLibrary
Librarian
User
+find:void
<<role>>
Librarian
accessLevel:String
name:String
librarianID:String
0..*
<<description>>
Title
title:String
author:String
publisher:String
+getTitle:String
+setTitle:void
Sequence Diagram
Library.findByKeywords
actor
<initial>


Modeling - Library Use Case - Eclipse SDK
File Edit Source Refactor Navigate Search Project Model Diagram Run Window Help
Model Navigator
Navigator
Library
Library Use Case
Book Checkout Sequence
User.java
Palette
Use Case Di...
Notes
Library Use Case
Library
Customer
Librarian
Library System
Check Out Book
Find Title by Author
Find Title by Keywords
Librarian
Association Link
Association Link
Association Link
accessLevel
librarianID
name
Library
findByAuthor
findByKeywords
findByTitle
Association Link
Association Link
Association Link
Association Link
Association Link
Association Link
address
branchNumber
hours
lnkLibrarian
phone
titleList
Title
getTitle
setTitle
Association Link
<<manager>>
Librarian
Library System
Find Title by Keywords
Find Title by Author
Check Out Book
Customer


Modeling - Book Checkout Sequence - Eclipse SDK
File Edit Source Refactor Navigate Search Project Model Diagram Run Window Help
Model Navigator
Navigator
name
Library
findByAuthor
findByKeywords
findByTitle
Association Link
Association Link
Association Link
Association Link
Association Link
Association Link
address
branchNumber
hours
InkLibrarian
phone
titleList
Title
getTitle
setTitle
Association Link
Association Link
Association Link
author
publisher
title
User
find
Library
Library Use Case
Book Checkout Sequence
User.java
Palette
Sequence Diagra...
Notes
Librarian
theLibrary:Library
user:User
1: float:=<undefined>
1.1: find(int ID):void


Use Case
Library.findByKeywords
<initial>:Library
actor
1: findByKeywords():void


Use Case
Library.findByKeywords
<initial>:Library
1: findByKeywords():void
<<actor>>
actor